

JobAlign

AI-Powered Job & Skill Alignment Database Design Document

1. Introduction

This document defines the database design for JobAlign. It records the tables, columns, keys, relationships, indexes and constraints that implement the requirements set out in the Requirement Analysis, and states the reasoning behind each significant design decision.

The schema is organised into six domains that correspond to the twelve system modules identified in the Requirement Analysis. It supports three user roles — Candidate, Recruiter and Administrator — together with an external AI Service that supplies extraction, summarization and feedback. The target engine is Microsoft SQL Server, accessed through Entity Framework Core.

2. Design Principles

Seven decisions shape the schema. Each traces to a stated requirement.

- One identity table, role-specific extensions. Users holds login and identity for every person. Candidates and Recruiters are 1:1 extension tables rather than separate silos, so authentication and role-based authorization are centralized.
- Original text is never overwritten. JobPostings.RawText is retained permanently alongside the normalized fields, so an extraction can always be reviewed against its source and re-run without loss.
- AI output is stored separately from the posting. JobExtractions is a nullable-until-processed table rather than columns bolted onto JobPostings, so a posting saves successfully even when the AI service fails.
- Every AI-derived value carries a confidence score. JobExtractions, JobSkills and Resumes each record confidence so that low-confidence results can be flagged for review.
- A skill exists exactly once. Skills is the canonical list and SkillAliases maps alternate terms onto it, so “C#”, “C Sharp” and “C-Sharp” resolve to one SkillId. New aliases are added as rows, never as schema changes.
- Analyses are stored as immutable snapshots. JobAnalyses and its child tables record the result of one analysis run, preserving history even after the candidate profile changes.
- Deletion is supported by design. Cascading deletes from Users through Candidates and their child tables allow a user to remove their own data on request.

3. Schema Overview

The twelve system modules map onto six data domains as follows.

Domain	Modules Served	Tables
Identity and Access	1. Authentication & User Management	Roles, Users
Organizations and Recruiters	3. Job Posting Management	Organizations, Recruiters
Candidate Profile	2. Candidate Profile Management, 7. Resume Analysis	Candidates, Education, Experience, Projects, Certifications, Resumes

Domain	Modules Served	Tables
Skill	5. Normalization, 6. Skill & Alias Management	SkillCategories, Skills, SkillAliases, CandidateSkills
Job Posting and Extraction	3. Job Posting Management, 4. AI Extraction	JobPostings, JobExtractions, JobSkills, SavedJobs
Matching and Feedback	8. Job Matching, 9. Skill Gap, 10. AI Feedback, 11. Comparison	JobAnalyses, AnalysisSkills, AnalysisFeedback, JobComparisons
Administration	12. Dashboard & Reports	AuditLogs

4. Table Definitions

PK denotes primary key, FK foreign key and UQ a unique constraint. All primary keys are integer surrogate keys generated by the database.

4.1 Identity and Access

Roles

Column	Type	Constraints / Notes
RoleId	int	PK, IDENTITY
RoleName	nvarchar(30)	NOT NULL, UQ — Candidate, Recruiter or Administrator
Description	nvarchar(256)	NULL

Users — one row per person, any role (FR-01, FR-13)

Column	Type	Constraints / Notes
UserId	int	PK, IDENTITY
FullName	nvarchar(120)	NOT NULL
Email	nvarchar(150)	NOT NULL, UQ — login identifier
PasswordHash	nvarchar(255)	NOT NULL — salted hash, never plain text (NFR-04)
RoleId	int	FK → Roles.RoleId, NO ACTION
IsActive	bit	NOT NULL, DEFAULT 1 — deactivation instead of deletion (FR-13)
CreatedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()
UpdatedAt	datetime2	NULL

4.2 Organizations and Recruiters

Organizations

Column	Type	Constraints / Notes
OrganizationId	int	PK, IDENTITY
Name	nvarchar(150)	NOT NULL
Industry	nvarchar(100)	NULL
Website	nvarchar(255)	NULL

Recruiters — role extension of Users (FR-09)

Column	Type	Constraints / Notes
RecruiterId	int	PK, IDENTITY
UserId	int	FK → Users.UserId, UQ, ON DELETE CASCADE
OrganizationId	int	FK → Organizations.OrganizationId, NULL
Title	nvarchar(100)	NULL

4.3 Candidate Profile

Candidates — role extension of Users (FR-01)

Column	Type	Constraints / Notes
CandidateId	int	PK, IDENTITY
UserId	int	FK → Users.UserId, UQ, ON DELETE CASCADE
Headline	nvarchar(150)	NULL
Location	nvarchar(100)	NULL
PhoneNumber	nvarchar(20)	NULL
Bio	nvarchar(max)	NULL
UpdatedAt	datetime2	NULL

The four tables below are child tables of Candidates, all with ON DELETE CASCADE. Together they satisfy FR-02.

Education

Column	Type	Constraints / Notes
EducationId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
Institution	nvarchar(150)	NOT NULL
Degree	nvarchar(100)	NULL
FieldOfStudy	nvarchar(100)	NULL
StartDate	date	NULL
EndDate	date	NULL — NULL indicates ongoing study

Experience

Column	Type	Constraints / Notes
ExperienceId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
CompanyName	nvarchar(150)	NOT NULL
JobTitle	nvarchar(150)	NOT NULL
StartDate	date	NULL
EndDate	date	NULL — NULL indicates the current role
Description	nvarchar(max)	NULL

Projects

Column	Type	Constraints / Notes
ProjectId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
Title	nvarchar(150)	NOT NULL

Column	Type	Constraints / Notes
Description	nvarchar(max)	NULL
ProjectUrl	nvarchar(255)	NULL

Certifications

Column	Type	Constraints / Notes
CertificationId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
Name	nvarchar(150)	NOT NULL
IssuingOrganization	nvarchar(150)	NULL
IssueDate	date	NULL
ExpiryDate	date	NULL

Resumes (FR-03, FR-18)

Column	Type	Constraints / Notes
Resumeld	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
FileUrl	nvarchar(255)	NOT NULL — stored file location, not the file itself
FileName	nvarchar(150)	NULL
UploadedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()
ParseStatus	nvarchar(20)	NOT NULL — Pending, Parsed or Failed
ExtractedData	nvarchar(max)	NULL — JSON returned by the AI service
ParseConfidence	decimal(4,3)	NULL — confidence indicator (NFR-06)

The resume file is held outside the database and referenced by path. Only extracted text is transmitted to the AI service, which satisfies the data-minimisation requirement of NFR-08.

4.4 Skill and Alias Management

SkillCategories

Column	Type	Constraints / Notes
CategoryId	int	PK, IDENTITY
CategoryName	nvarchar(80)	NOT NULL, UQ — e.g. Language, Framework, Cloud

Skills — the canonical skill list (FR-05, FR-14)

Column	Type	Constraints / Notes
SkillId	int	PK, IDENTITY
SkillName	nvarchar(100)	NOT NULL, UQ — canonical name, e.g. “C#”
CategoryId	int	FK → SkillCategories.CategoryId, NULL
IsActive	bit	NOT NULL, DEFAULT 1 — deactivated rather than deleted
CreatedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()

SkillAliases — alternate terms mapped to one canonical skill

Column	Type	Constraints / Notes
AliasId	int	PK, IDENTITY
SkillId	int	FK → Skills.SkillId, CASCADE
AliasName	nvarchar(100)	NOT NULL, UQ — e.g. “C Sharp” maps to “C#”

Column	Type	Constraints / Notes
CreatedByUserId	int	FK → Users.UserId, NULL — NULL indicates system-generated

SkillAliases is the mechanism that makes normalization work generically. A newly discovered variant is inserted as a row, so no schema change or code deployment is required to recognise it.

CandidateSkills — junction table (FR-02)

Column	Type	Constraints / Notes
CandidateSkillId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
SkillId	int	FK → Skills.SkillId, NO ACTION
ProficiencyLevel	nvarchar(20)	NOT NULL — see Section 7
YearsExperience	decimal(4,1)	NULL
Source	nvarchar(20)	NOT NULL — Manual or Resume
UQ(CandidateId, SkillId)	constraint	Prevents the same skill being recorded twice

4.5 Job Posting and AI Extraction

JobPostings — recruiter source of truth (FR-09, FR-10)

Column	Type	Constraints / Notes
JobId	int	PK, IDENTITY
RecruiterId	int	FK → Recruiters.RecruiterId, NO ACTION
RawText	nvarchar(max)	NOT NULL — original posting, never overwritten (NFR-07)
NormalizedTitle	nvarchar(150)	NULL — standardized job title (FR-05)
Location	nvarchar(100)	NULL — normalized location
ExperienceMinYears	decimal(4,1)	NULL — NULL means not stated, never zero
ExperienceMaxYears	decimal(4,1)	NULL
SalaryMin	decimal(18,2)	NULL
SalaryMax	decimal(18,2)	NULL
SalaryCurrency	nvarchar(8)	NULL
CompletenessScore	decimal(4,3)	NULL — supports the completeness review in FR-11
Status	nvarchar(20)	NOT NULL — see Section 7 (FR-15)
PostedAt	datetime2	NOT NULL
UpdatedAt	datetime2	NULL

Extracted values are nullable throughout. A posting that does not state a salary stores NULL, which remains distinguishable from a genuine value of zero and keeps the posting out of salary-based sorting.

JobExtractions — one AI pass per posting (FR-17, FR-19, NFR-05, NFR-06)

Column	Type	Constraints / Notes
ExtractionId	int	PK, IDENTITY
JobId	int	FK → JobPostings.JobId, UQ, CASCADE
ExtractedJson	nvarchar(max)	NULL — raw AI response, retained for audit
SummaryText	nvarchar(max)	NULL — concise job summary (FR-19)
ConfidenceScore	decimal(4,3)	NULL — overall extraction confidence (NFR-06)
Status	nvarchar(20)	NOT NULL — Pending, Completed or Failed
ErrorMessage	nvarchar(1024)	NULL — reason a failed extraction can be retried

Column	Type	Constraints / Notes
ExtractedAt	datetime2	NULL
ReviewedByUserId	int	FK → Users.UserId, NULL — recruiter who confirmed the result (FR-11)

This table exists separately from JobPostings so that a posting is saved and readable whether or not the AI service responded. That separation is the direct implementation of NFR-05.

JobSkills — junction table (FR-12)

Column	Type	Constraints / Notes
JobSkillId	int	PK, IDENTITY
JobId	int	FK → JobPostings.JobId, CASCADE
SkillId	int	FK → Skills.SkillId, NO ACTION
RequirementType	nvarchar(20)	NOT NULL — Required or Preferred
RawTerm	nvarchar(100)	NULL — the term as written, retained for review
ConfidenceScore	decimal(4,3)	NULL (NFR-06)
UQ(JobId, SkillId)	constraint	Prevents a duplicate skill on one posting

SavedJobs — junction table (FR-04)

Column	Type	Constraints / Notes
SavedJobId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
JobId	int	FK → JobPostings.JobId, CASCADE
SavedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()
UQ(CandidateId, JobId)	constraint	A candidate saves a posting at most once

4.6 Matching, Skill Gap and Feedback

JobAnalyses — one immutable snapshot per run (FR-06, FR-08)

Column	Type	Constraints / Notes
AnalysisId	int	PK, IDENTITY
CandidateId	int	FK → Candidates.CandidateId, CASCADE
JobId	int	FK → JobPostings.JobId, NO ACTION
MatchScore	decimal(5,2)	NOT NULL — overall job-fit percentage, e.g. 80.00
RequiredSkillScore	decimal(5,2)	NULL — required-skill component
PreferredSkillScore	decimal(5,2)	NULL — preferred-skill component
ExperienceScore	decimal(5,2)	NULL — experience component
AnalyzedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()

Scores are stored rather than recalculated on read. This satisfies the two-second load requirement of NFR-02 and preserves the analysis history that FR-08 depends on: a result recomputed after the profile changed would no longer describe the analysis the candidate originally saw.

AnalysisSkills — matched and missing skills for one analysis

Column	Type	Constraints / Notes
AnalysisSkillId	int	PK, IDENTITY
AnalysisId	int	FK → JobAnalyses.AnalysisId, CASCADE
SkillId	int	FK → Skills.SkillId, NO ACTION

Column	Type	Constraints / Notes
SkillStatus	nvarchar(20)	NOT NULL — Matched or Missing
RequirementType	nvarchar(20)	NOT NULL — Required or Preferred at the time of analysis
UQ(AnalysisId, SkillId)	constraint	A skill appears once per analysis

Matched and missing skills share one table with a status column rather than occupying two near-identical tables. The structure and the queries are the same in both cases, so separating them would add a table without adding meaning.

AnalysisFeedback (FR-07, FR-20)

Column	Type	Constraints / Notes
FeedbackId	int	PK, IDENTITY
AnalysisId	int	FK → JobAnalyses.AnalysisId, UQ, CASCADE
FeedbackText	nvarchar(max)	NOT NULL — AI-generated strengths and recommendations
GeneratedAt	datetime2	NOT NULL

JobComparisons and JobComparisonItems (FR-08)

Column	Type	Constraints / Notes
ComparisonId	int	PK, IDENTITY — JobComparisons
CandidateId	int	FK → Candidates.CandidateId, CASCADE
CreatedAt	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()
ItemId	int	PK, IDENTITY — JobComparisonItems
ComparisonId	int	FK → JobComparisons.ComparisonId, CASCADE
AnalysisId	int	FK → JobAnalyses.AnalysisId, NO ACTION

A comparison references analyses rather than postings, so the comparison shows the scores as they stood when each posting was analysed.

4.7 Administration

AuditLogs (FR-13, FR-15, FR-16, NFR-07)

Column	Type	Constraints / Notes
LogId	bigint	PK, IDENTITY
UserId	int	FK → Users.UserId, NULL — NULL denotes a system action
Action	nvarchar(100)	NOT NULL — e.g. skill.merged, job.flagged
EntityType	nvarchar(50)	NOT NULL — e.g. JobPosting, Skill
EntityId	int	NULL
Details	nvarchar(max)	NULL — free-form context
Timestamp	datetime2	NOT NULL, DEFAULT SYSUTCDATETIME()

5. Relationship Summary

Relationship	Cardinality	Delete Rule and Rationale
Roles → Users	1 : M	NO ACTION — a role in use cannot be removed
Users → Candidates / Recruiters	1 : 1	CASCADE — role profile has no meaning without the user
Organizations → Recruiters	1 : M	NO ACTION
Recruiters → JobPostings	1 : M	NO ACTION — postings survive recruiter departure

Relationship	Cardinality	Delete Rule and Rationale
Candidates → Education, Experience, Projects, Certifications, Resumes	1 : M each	CASCADE — supports user-initiated deletion (NFR-08)
SkillCategories → Skills	1 : M	NO ACTION
Skills → SkillAliases	1 : M	CASCADE
Candidates ↔ Skills	M : N	Resolved through CandidateSkills
JobPostings ↔ Skills	M : N	Resolved through JobSkills
Candidates ↔ JobPostings	M : N	Resolved through SavedJobs
JobPostings → JobExtractions	1 : 1	CASCADE
Candidates, JobPostings → JobAnalyses	M : 1 each	CASCADE on Candidates, NO ACTION on JobPostings
JobAnalyses → AnalysisSkills	1 : M	CASCADE
JobAnalyses → AnalysisFeedback	1 : 1	CASCADE
JobComparisons → JobComparisonItems → JobAnalyses	1 : M, M : 1	CASCADE then NO ACTION
Users → AuditLogs	1 : M	NO ACTION — audit history outlives the account

Two cascade paths would otherwise reach JobAnalyses, one through Candidates and one through JobPostings. SQL Server rejects multiple cascade paths to the same table, so the posting relationship uses NO ACTION and the candidate relationship carries the cascade.

6. Indexing Strategy

Indexing targets NFR-01, which requires analysis to complete within ten seconds, and NFR-02, which requires the saved-jobs list to load within two seconds for up to 500 postings.

Index	Type	Purpose
UQ_Users_Email	Unique	Login lookup and duplicate account prevention
UQ_Skills_SkillName	Unique	Guarantees one row per canonical skill (FR-14)
UQ_SkillAliases_AliasName	Unique	Prevents one alias resolving to two skills
UQ_CandidateSkills_Candidate_Skill	Unique composite	Prevents duplicate skills on a profile
UQ_JobSkills_Job_Skill	Unique composite	Prevents duplicate skills on a posting
UQ_SavedJobs_Candidate_Job	Unique composite	A posting is saved once per candidate
IX_JobAnalyses_Candidate_Job	Composite	Job-fit lookup and comparison history (FR-06, FR-08)
IX_SavedJobs_CandidateId	Non-clustered	Saved-jobs list load (NFR-02)
IX_JobPostings_Status	Non-clustered	Moderation queue for flagged postings (FR-15)
IX_AnalysisSkills_SkillId	Non-clustered	Skill-gap statistics across candidates (FR-16)
IX_JobSkills_SkillId	Non-clustered	Reverse lookup from skill to postings

The composite unique indexes carry the integrity rules that matter most in daily use. Without them, re-running an extraction could insert a second copy of the same skill against a posting and silently distort every score derived from it.

7. Enumerated Values

Status columns are stored as short text rather than integers so that the database remains readable during testing and review. Permitted values are enforced in the application layer.

Table	Column	Permitted Values
Roles	RoleName	Candidate, Recruiter, Administrator
JobPostings	Status	Draft, Published, Flagged, Archived
JobExtractions	Status	Pending, Completed, Failed
Resumes	ParseStatus	Pending, Parsed, Failed
JobSkills	RequirementType	Required, Preferred
CandidateSkills	ProficiencyLevel	Beginner, Intermediate, Advanced, Expert
CandidateSkills	Source	Manual, Resume
AnalysisSkills	SkillStatus	Matched, Missing

8. Normalization

The schema is in third normal form. Every non-key column depends on its table's primary key alone, and no repeating groups or multi-valued columns exist. Skills are held in junction tables rather than as delimited strings; SkillCategories and Organizations are separate tables so that a category or company name is stored once; and CandidateSkills, JobSkills, SavedJobs, AnalysisSkills and JobComparisonItems each resolve a many-to-many relationship.

Junction tables carry their own attributes, which is why they are tables rather than simple link rows. CandidateSkills records proficiency and source, and JobSkills records whether a skill is required or preferred together with its extraction confidence. That information belongs to the relationship, not to either entity.

Two deliberate exceptions are recorded here so they are not mistaken for oversights. JobPostings.CompletenessScore is cached rather than derived on every read, because the recruiter review screen in FR-11 would otherwise recompute it per posting. JobAnalyses likewise stores computed scores, both for NFR-02 and because an analysis is a historical record that must not change when the underlying profile does.

JobExtractions.ExtractedJson and Resumes.ExtractedData remain unstructured JSON. The shape of AI output evolves, and holding it as text allows the extraction contract to change without a schema migration, which supports NFR-10 and NFR-11.

9. Requirement Traceability

Ref	Requirement	Implementation
FR-01	Register, log in, manage profile	Users, Roles, Candidates
FR-02	Manage skills, experience, education, projects, certifications	CandidateSkills, Experience, Education, Projects, Certifications
FR-03	Upload and analyze a resume	Resumes
FR-04	Save and analyze job postings	SavedJobs, JobAnalyses
FR-05	View normalized job details and skills	JobPostings.NormalizedTitle, Skills, SkillAliases
FR-06	View job-fit score, matched skills and gaps	JobAnalyses, AnalysisSkills
FR-07	Receive AI feedback	AnalysisFeedback
FR-08	Compare postings and view history	JobComparisons, JobComparisonItems

Ref	Requirement	Implementation
FR-09, FR-10	Create and manage postings; view extracted detail	JobPostings, Recruiters, JobExtractions
FR-11	Review completeness and edit extracted detail	CompletenessScore, JobExtractions.ReviewedByUserId
FR-12	Manage required and preferred skills	JobSkills.RequirementType
FR-13	Manage users, roles and access	Users.RoleId, Users.IsActive, AuditLogs
FR-14	Manage skills, aliases and categories	Skills, SkillAliases, SkillCategories
FR-15	Manage duplicate or inappropriate postings	JobPostings.Status, AuditLogs
FR-16	System reports and skill-gap statistics	AnalysisSkills aggregation, AuditLogs
FR-17	Extract job details and skills	JobExtractions, JobSkills
FR-18	Extract information from resumes	Resumes.ExtractedData
FR-19	Generate concise job summaries	JobExtractions.SummaryText
FR-20	Generate skill-gap feedback	AnalysisFeedback
NFR-02	Fast saved-job list	Stored scores; IX_SavedJobs_CandidateId
NFR-04	Hashed passwords	Users.PasswordHash
NFR-05	AI failure does not block saving	JobExtractions separate from JobPostings
NFR-06	Confidence indicators	ConfidenceScore on JobExtractions, JobSkills, Resumes
NFR-07	Original text retained	JobPostings.RawText, AuditLogs
NFR-08	Data minimisation and deletion	Cascading deletes from Users and Candidates